PrimeWave Resource Reservation System – Developer Guide

Prepared for S.P. Sachintha Nimesh · PrimeWave Lab SE Internship Technical Assessment · Deadline: Thu 15 Oct 2026, 11:59 PM (Sri Lanka time)

1. Read This First: Time Plan

The deadline is **today**. The form closes at midnight and late submissions are rejected, so plan backwards and aim to submit by about 10:30 PM to leave a safety buffer.

Block	Goal
0:00–0:20	Setup: repo, Next.js, Postgres, Prisma
0:20–0:50	Schema, migration, seed data, ER diagram
0:50–2:30	Core backend: resources, reservations, conflict check
2:30–4:00	UI: resource list, booking form, calendar/list view
4:00–4:45	Auth (simple), validation, error handling, polish
4:45–5:30	README, tests for conflict logic, deploy check
5:30–6:00	Demo video (3–5 min), submit form

If you have less time than this, cut from the bottom of the "Nice to have" list in section 3, never from the core.

2. What the Assessors Actually Want

The brief gives a business scenario, not a spec. The evaluation criteria tell you what matters:

- **Interpretation of the problem:** the key pain points are *overlapping reservations*, *uncertain availability* and *manual records*. Your app must visibly solve these.
- **Engineering judgement:** prevent double booking at the database level, not just in the UI.

- **Reliability:** "remain reliable as day-to-day activities grow" hints at concurrency safety, indexes and pagination.
- **Architecture and DB design:** clean schema, proper constraints, sensible relationships.
- **Code quality:** typed, layered, readable, small functions.
- **UX:** easy booking, clear availability, helpful error messages.
- **Communication:** README, ER diagram and demo video explain your decisions.

The brief also says AI tools are allowed, but you must be able to **explain every decision**. Understand all code you keep.

3. Scope Decision (Prioritised)

Must have (core)

1. **Resources:** create, edit, list, deactivate (name, type/category, location, capacity, description, active flag).
2. **Reservations:** create, view, cancel, with title, resource, start time, end time, reserved by, optional notes.
3. **Conflict prevention:** reject any reservation overlapping an existing active reservation for the same resource. Enforced in the database.
4. **Availability view:** for a chosen resource and date, show booked and free slots. A day or week calendar/timeline view is ideal.
5. **Filtering and search:** by resource, category, date range, status.
6. **Validation:** end after start, no past bookings, required fields, sensible max duration.
7. **Persistent storage** in PostgreSQL, runnable locally.

Should have

8. **Simple authentication and roles:** `ADMIN` (manages resources, sees all reservations) and `USER` (books and cancels own). Keep it simple: credentials login with hashed passwords and a session cookie.
9. **Usage overview:** a small dashboard with today's reservations, upcoming bookings and utilisation per resource.
10. **Status lifecycle:** `CONFIRMED`, `CANCELLED` (soft delete, keep history).

Nice to have (cut first)

- Recurring reservations
- Email notifications
- Approval workflow
- Drag-and-drop calendar
- Audit log

Rule: a working, reliable core with clean code beats half-built extras. The brief explicitly says it values prioritisation over complexity.

4. Recommended Tech Stack

Required by the brief: **Next.js (App Router), TypeScript, PostgreSQL, Git and GitHub.**

Recommended additions:

Concern	Choice	Why
ORM and migrations	**Prisma**	Fast to set up, typed, migrations built in, easy to explain
Validation	**Zod**	Share schemas between client and server
UI	**Tailwind CSS + shadcn/ui**	Fast, clean, accessible
Auth	**Auth.js (NextAuth) credentials** or a simple JWT cookie with `jose` + `bcryptjs`	Keep it light
Dates	**date-fns** (or `dayjs`)	Simple, reliable
Forms	**react-hook-form** + Zod resolver	Less boilerplate
Tests	**Vitest**	Test the conflict logic
Local DB	**Local PostgreSQL 18** with `psql`	Already installed on your machine; Docker is optional

If Prisma feels risky for the overlap constraint (see section 6), you can use raw SQL for that single migration. Prisma supports this through custom migration files.

5. Architecture

Keep it a single Next.js app with clear layers:

```
src/
  app/
    (auth)/login/page.tsx
    (app)/
      dashboard/page.tsx
      resources/page.tsx
      resources/[id]/page.tsx      # availability calendar
      reservations/page.tsx
      reservations/new/page.tsx
    api/
      resources/route.ts
      resources/[id]/route.ts
      reservations/route.ts
      reservations/[id]/route.ts
      availability/route.ts
  components/                       # UI pieces
  lib/
    db.ts                           # Prisma client singleton
    auth.ts                         # session helpers
    validators/                     # Zod schemas
    services/
      resource.service.ts
      reservation.service.ts        # business rules live here
    errors.ts                       # AppError, ConflictError, etc.
prisma/
  schema.prisma
  migrations/
  seed.ts
docs/
  er-diagram.png
```

Layering principle: route handlers (or server actions) only parse input, call a service and map errors to HTTP responses. All business rules live in `services/`. This is easy to test and easy to explain in the video.

Server Actions vs Route Handlers: either is fine. Route Handlers give you a clean REST API that is easy to demo and test. Server Actions are shorter for forms. Pick one style and stay consistent. Recommendation: Route Handlers for the API plus server components for reading data.

6. Database Design

Entities

User

- `id` (uuid, PK), `name`, `email` (unique), `passwordHash`, `role` (ADMIN | USER), `createdAt`

ResourceCategory *(optional but nice: Room, Equipment, Vehicle, Lab)*

- `id`, `name` (unique)

Resource

- `id` (uuid, PK), `name`, `categoryId` (FK), `location`, `capacity` (nullable), `description`, `isActive` (default true), `createdAt`, `updatedAt`

Reservation

- `id` (uuid, PK), `resourceId` (FK), `userId` (FK), `title`, `notes`, `startTime` (timestamptz), `endTime` (timestamptz), `status` (CONFIRMED | CANCELLED), `createdAt`, `updatedAt`, `cancelledAt` (nullable)

Relationships

- A Category has many Resources.
- A Resource has many Reservations.
- A User has many Reservations.

Constraints and indexes (this is where you score points)

1. `CHECK (end_time > start_time)`
2. **Prevent overlaps at the database level** using a PostgreSQL exclusion constraint:

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

ALTER TABLE "Reservation"
ADD CONSTRAINT no_overlapping_reservations
EXCLUDE USING gist (
  "resourceId" WITH =,
  tstzrange("startTime", "endTime", '[)') WITH &&
)
WHERE (status = 'CONFIRMED');
```

This guarantees no two confirmed reservations overlap for the same resource, even under concurrent requests. Half-open ranges `[)` mean a booking ending at 10:00 does not clash with one starting at 10:00.

3. Index on `(resourceId, startTime)` for availability queries.
4. Index on `(userId, startTime)` for "my reservations".
5. Unique index on `User.email`.

Fallback if the exclusion constraint gives trouble: wrap the check and insert in a serializable transaction (or lock the resource row with `SELECT ... FOR UPDATE`) and check for overlap with `startTime < newEnd AND endTime > newStart`. Mention the trade-off in your README.

Time zones

Store all timestamps as UTC (`timestamptz`). Display in the user's local time (Asia/Colombo for this audience). State this decision in the README.

7. Core Business Logic

Overlap rule

Two reservations on the same resource overlap when:

```
existingStart < newEnd  AND  existingEnd > newStart
```

Back-to-back bookings (one ends exactly when another starts) are allowed.

Create reservation flow

1. Authenticate the user.

2. Validate input with Zod (end > start, start not in the past, duration within limits).
3. Confirm the resource exists and `isActive`.
4. Insert inside a transaction.
5. If the database raises the exclusion violation (Postgres error code `23P01`), catch it and return **409 Conflict** with a friendly message and, ideally, the next free slot.
6. Return the created reservation.

Cancel flow

- Owner or admin only.
- Set `status = CANCELLED` and `cancelledAt = now()`. Do not hard-delete.
- Cancelled bookings free the slot automatically because the constraint only covers `CONFIRMED`.

Edit/reschedule flow

- Same validation and conflict handling as create. Exclude the reservation's own id when checking overlap (the constraint handles this naturally on UPDATE).

Availability query

For a resource and date range, fetch confirmed reservations ordered by `startTime`, then compute free gaps within business hours (for example 08:00–18:00, configurable via an environment variable or constant).

Deactivating a resource

Block new bookings. Decide what to do with future bookings (recommended: warn the admin and require cancelling them first). Document your choice.

8. API Design (REST)

Method	Path	Purpose
GET	`/api/resources`	List, filter by category, search, paginate
POST	`/api/resources`	Create (admin)

Method	Path	Purpose
GET	`/api/resources/:id`	Detail
PATCH	`/api/resources/:id`	Update or deactivate (admin)
GET	`/api/availability?resourceId=&date=`	Booked and free slots
GET	`/api/reservations`	List with filters (resource, status, from, to, mine)
POST	`/api/reservations`	Create
PATCH	`/api/reservations/:id`	Reschedule
POST	`/api/reservations/:id/cancel`	Cancel

Conventions:

- JSON errors in one shape: `{ "error": { "code": "CONFLICT", "message": "..." } }`
- Status codes: 200, 201, 400 (validation), 401, 403, 404, 409 (conflict), 500.
- Pagination with `page` and `pageSize` (default 20, max 100).

9. UI Plan

1. **Login page** (and a seeded demo admin and user).
2. **Dashboard:** today's reservations, upcoming list, quick stats (total resources, bookings today, utilisation).
3. **Resources page:** card or table list with category filter, search, "Book" button; admin sees add/edit/deactivate.
4. **Resource detail with availability:** a day or week timeline. Booked blocks in a clear colour, free time visible, click a free slot to prefill the booking form.
5. **Booking form:** resource, date, start, end, title, notes. Show inline conflict errors and suggest alternatives.
6. **My reservations / All reservations:** table with status badges, filters, cancel button with a confirm dialog.

UX details that impress: loading and empty states, toast notifications, disabled submit while saving, responsive layout, accessible labels, clear error text such as "Conference Room A is already booked 10:00–11:30 by another user. Next free slot: 11:30."

10. Step-by-Step Setup Commands

```
# 1. Create project
npx create-next-app@latest resource-reservation \
  --typescript --tailwind --eslint --app --src-dir --import-alias "@/*"
cd resource-reservation

# 2. Git
git init && git add . && git commit -m "chore: initial Next.js app"

# 3. Dependencies
npm i prisma @prisma/client zod react-hook-form @hookform/resolvers \
  date-fns bcryptjs jose
npm i -D vitest tsx @types/bcryptjs
npx prisma init

# 4. UI kit (optional)
npx shadcn@latest init
```

Local PostgreSQL 18 (Windows). Create the database once, using the password of your `postgres` user. Docker is optional, so no `docker-compose.yml` is needed.

```
set PATH=%PATH%;C:\Program Files\PostgreSQL\18\bin
psql -U postgres -h localhost -c "CREATE DATABASE reservations;"
```

.env (local only, never commit it; make sure `.env` is listed in `.gitignore`):

```
DATABASE_URL="postgresql://postgres:123123@localhost:5432/reservations"
AUTH_SECRET="change-me"
```

.env.example (commit this one, with placeholders instead of the real password):

```
DATABASE_URL="postgresql://postgres:YOUR_PASSWORD@localhost:5432/reservations
"
AUTH_SECRET="change-me"
```

Prisma workflow

```
npx prisma migrate dev --name init
# then create a custom migration for the exclusion constraint:
npx prisma migrate dev --create-only --name add_overlap_constraint
# paste the SQL from section 6 into the generated migration.sql, then:
npx prisma migrate dev
npx prisma db seed
npm run dev
```

Add to `package.json`:

```
"prisma": { "seed": "tsx prisma/seed.ts" }
```

11. Git Workflow

- Create the GitHub repo **now** and push early. Make it public, or private with access shared if the brief's reviewers need it (public is safest, and the brief requires the repo be accessible to the review team).
- Small, meaningful commits using conventional messages: `feat: add reservation conflict check`, `fix: handle timezone offset`, `docs: add README`.
- Reviewers read commit history, so avoid one giant final commit.
- Use `main` plus short feature branches if you have time, otherwise commit straight to `main`.
- Add `.gitignore` entries for `.env`, `node_modules`, `.next`.

12. Testing

You do not need full coverage. Test what matters most:

1. **Overlap logic unit tests (Vitest):** exact overlap, partial overlap start and end, contained, containing, back-to-back (allowed), different resources (allowed), cancelled reservation (allowed).
2. **Validation tests:** end before start, past start, over max duration.
3. **One concurrency test or manual proof:** fire two simultaneous POSTs for the same slot and show only one succeeds. This is a great demo moment.

13. README Checklist

The brief says another developer must be able to understand, set up and run it. Include:

1. Project overview and the business problem solved
2. Features list
3. Tech stack and why
4. Architecture overview (folder structure, layers)
5. Prerequisites (Node version, PostgreSQL 18, Docker optional)
6. Setup steps (clone, `.env`, `create the database`, migrate, seed, run)
7. Demo credentials from the seed
8. API summary
9. Key design decisions: DB-level overlap prevention, UTC storage, soft cancellation, role model
10. Assumptions and limitations
11. Future improvements (recurring bookings, notifications, approvals)
12. Link to the ER diagram
13. How to run tests

14. ER Diagram

The brief requires a **shareable link** to the ER diagram, submitted via the form.

- Fastest options: **dbdiagram.io** (write DBML, share link), **drawSQL**, **Mermaid** in the README, or draw.io with a public link.
- Include all entities, PK/FK, cardinalities and key constraints.
- Make sure the link is set to **anyone with the link can view**. Test it in an incognito window.

Starter DBML for dbdiagram.io:

```
Table users {
  id uuid [pk]
  name varchar
  email varchar [unique]
  password_hash varchar
  role varchar
  created_at timestamptz
```

```
}
Table resource_categories {
  id uuid [pk]
  name varchar [unique]
}
Table resources {
  id uuid [pk]
  name varchar
  category_id uuid [ref: > resource_categories.id]
  location varchar
  capacity int
  description text
  is_active boolean
  created_at timestamptz
  updated_at timestamptz
}
Table reservations {
  id uuid [pk]
  resource_id uuid [ref: > resources.id]
  user_id uuid [ref: > users.id]
  title varchar
  notes text
  start_time timestamptz
  end_time timestamptz
  status varchar
  created_at timestamptz
  updated_at timestamptz
  cancelled_at timestamptz
}
```

15. Demo Video Script (3–5 minutes)

Record with OBS, Loom or your OS recorder. Rehearse once.

Time	Content
0:00–0:30	Introduce the problem: double bookings, manual records, unclear availability. State your solution in one sentence.

Time	Content
0:30–1:30	Walk through the app: log in as admin, add a resource, view the dashboard.
1:30–2:30	Book a resource, view availability timeline, attempt an overlapping booking and show the conflict message.
2:30–3:15	Cancel a booking, show the slot freed, show filters and search. Optionally show role differences.
3:15–4:30	Technical decisions: Next.js App Router layering, Prisma schema and ER diagram, **exclusion constraint** for overlap safety, UTC storage, validation with Zod, tests.
4:30–5:00	Limitations and what you would do next.

Tip: speak clearly, keep the browser zoomed so text is readable, and upload as an unlisted YouTube or Google Drive link with view access for anyone with the link.

16. Submission Checklist

The deadline is **Thursday 15 October 2026, 11:59 PM Sri Lanka time**. The form closes at midnight and nothing is accepted afterward.

- [ ] GitHub repo is accessible to the review team (test in incognito)
- [ ] README complete and the setup steps verified on a clean clone
- [ ] `.env.example` included, no secrets committed
- [ ] Migrations and seed work from scratch
- [ ] Demo video 3–5 minutes, link permissions open
- [ ] ER diagram link opens without login
- [ ] Submission form filled with the **exact email address used to send your CV**
- [ ] Submitted before about 10:30 PM, with a screenshot of the confirmation

Official form: https://forms.gle/HzQH6tB5c4edPjnn8

17. Common Pitfalls to Avoid

- Checking overlap only in the UI or only in application code (race conditions).
- Mixing time zones, or storing local time without an offset.
- Using a strict `<=` comparison that wrongly blocks back-to-back bookings.
- Hard-deleting cancelled reservations, which loses history.
- Building too many features and leaving the core buggy.
- Leaving the repo or video private.
- Committing `.env`.
- Using code from an AI tool that you cannot explain.
- Leaving recording and submission to the last 15 minutes.

18. Questions You Should Be Ready to Answer

1. Why did you choose this schema, and how does it scale with more resources and reservations?
2. How do you prevent double booking under concurrent requests?
3. Why store timestamps in UTC?
4. Why soft-cancel instead of delete?
5. How would you add recurring reservations or approvals?
6. How would you deploy this to production (for example AWS with RDS, or Vercel plus managed Postgres)?
7. What would you improve with more time?

19. Next Step

Start with section 10 (setup), then the schema in section 6. Once the project is scaffolded, I can help you write the Prisma schema, the reservation service with conflict handling, the seed script, or the UI components one at a time.